Analyzing the Survey of Consumer Finances

1 Introduction

The Survey of Consumer Finances (SCF) is a triennial survey of U.S. household finances conducted by the Federal Reserve Board. It is among the most detailed data sources on U.S. household wealth, income, and financial behavior. Valid estimation from the SCF requires handling two design features:

1. Survey design: the SCF uses a dual-frame sample with 999 bootstrap replicate weights, which are used to estimate sampling variance.
2. Multiple imputation: the SCF fills in missing answers five times, giving five implicates. Each is a complete version of the data with different filled-in values.

The `scf` package handles both. It wraps the five implicates in replicate-weighted `survey::svrepdesign()` objects, runs each estimate in every implicate, and pools the results. See `?scf_variance` for how standard errors are computed.

Throughout, the unit of analysis is the SCF's primary economic unit: the person or couple at the economic center of the household and everyone financially dependent on them. The Fed calls this unit a "family"; this package calls it a household.

This vignette demonstrates the core workflow. It runs on the package's mock data, a small sample of the 2022 survey, so the numbers shown are not estimates for the population; the same code runs on the full data. For methodological background, see the package paper listed under Learn More.

2 Workflow

2.1 1. Downloading and Loading the Data

Download SCF data and load it into a multiply-imputed survey object using `scf_download()` and `scf_load()`.

```
scf_download(2022)
scf2022 <- scf_load(2022)
```

The result is a `scf_mi_survey` object containing five `svyrep.design` objects, one per implicate, and survey-year metadata. The package covers every wave from 1989 to 2025; `scf_download(2025)` fetches the latest. Each object records the dollar year of the Fed's summary variables (see Dollar values below):

```
scf2022
#> SCF Multiply-Imputed Survey Object
#> ----------------------------------
#> Year:           2022
#> Households (N): 5,793,387
#> Implicates:     5
#> Replicate weights per implicate: 999
#> Dollar year:    2022
scf2022$dollar_year
#> [1] 2022
```

2.2 2. Creating and Transforming Variables

`scf_update()` adds or modifies variables uniformly across all five implicates. Bottom-code skewed variables before logging to avoid `log(0)`.

```
scf2022 <- scf_update(scf2022,
  senior       = age >= 65,
  female       = factor(hhsex, levels = 1:2, labels = c("Male", "Female")),
  rich         = networth > 1e6,
  networth     = ifelse(networth > 1, networth, 1),
  log_networth = log(networth),
  income       = ifelse(income > 1, income, 1),
  log_income   = log(income)
)
```

Use `names(scf2022$mi_design[[1]]$variables)` to inspect available variables.

When a transformation depends on the distribution within each implicate, such as ranks, percentile flags, or group z-scores, use `scf_update_by_implicate()` instead:

```
scf2022 <- scf_update_by_implicate(scf2022, function(df) {
  df$wealth_rank <- rank(df$networth) / nrow(df)
  df
})
```

`scf_subset()` keeps the households that meet a condition, in every implicate. Estimates from a subset describe that group alone; to compare groups, use `by` in the estimating functions instead.

```
seniors <- scf_subset(scf2022, age >= 65)
scf_median(seniors, ~networth)
#> Multiply-Imputed Median Estimate
#>
#>  variable quantile estimate       se    min    max  n
#>  networth      0.5   179890 67650.09 152800 211350 54
```

2.2.1 Dollar values

Summary variables made by the Fed, such as `income`, `networth`, and `asset`, are in the dollars of one year in every survey year: 2025 for files downloaded since October 2026, 2022 before that. `scf_load()` records this as `dollar_year`. Raw survey variables, named `x` followed by a number, are in dollars of the survey year. Before comparing a raw variable across years, or combining it with summary variables, convert it with `scf_real()`. List income items, which are reported for the year before the survey, in `income`:

```
scf2016 <- scf_real(scf2016, ~x5702 + x3915, income = ~x5702)
```

To report a result in dollars of the survey year, as in the Fed's nominal-dollar tables, use `scf_nominal()` on the result. Both functions stop with an error if given the wrong kind of variable, unless `force = TRUE`.

2.3 3. Univariate and Bivariate Distributions

`scf_mean()`, `scf_median()`, and `scf_percentile()` return pooled population estimates with standard errors. The `by` argument produces group-level estimates.

```
scf_mean(scf2022, ~networth, by = ~senior)
#> Multiply-Imputed, Replicate-Weighted Mean Estimate
#>
#>  group variable  estimate       se     min       max   n
#>  FALSE networth  809777.2 123986.1  771184  821626.9 146
#>   TRUE networth 1595858.8 619767.0 1355046 1812726.0  54
scf_median(scf2022, ~income, by = ~female)
#> Multiply-Imputed Median Estimate
#>
#>   group variable quantile estimate       se      min      max   n
#>    Male   income      0.5 85824.40 6155.021 85392.03 86472.95 143
#>  Female   income      0.5 49721.94 5238.235 49721.94 49721.94  57
scf_percentile(scf2022, ~networth, q = 0.9)
#> SCF Percentile Estimate
#>
#>  variable quantile estimate       se     min     max   n
#>  networth      0.9  1197722 378276.9 1039600 1360000 200
scf_percentile(scf2022, ~networth, q = 0.75, by = ~female)
#> SCF Percentile Estimate
#>
#>   group variable quantile estimate        se    min    max   n
#>    Male networth     0.75   642660 188294.66 623800 698700 143
#>  Female networth     0.75   238504  51343.88 237680 241800  57
```

`scf_pctile_sum()` groups households by their position in a variable's distribution and summarizes each group, as the Fed's tables do by income and net worth percentile.

```
scf_pctile_sum(scf2022, ~networth, probs = c(0, 0.5, 0.9, 1),
               labels = c("bottom50", "50-90", "top10"),
               method = "stack", stat = "mean")
#>      group variable   estimate
#> 1 bottom50 networth   41993.99
#> 2    50-90 networth  446963.14
#> 3    top10 networth 7935229.58
```

Discrete distributions are summarized with `scf_freq()` and cross-tabulated with `scf_xtab()`.

```
scf_freq(scf2022, ~senior)
#> SCF Frequency Table (Pooled Results)
#>
#>  group category proportion se_proportion   n
#>    All    FALSE   75.77111      3.113888 200
#>    All     TRUE   24.22889      3.113888 200
scf_xtab(scf2022, ~senior, ~female, scale = "col")
#> SCF Cross-Tabulation
#> Row Variable: senior | Column Variable: female
#> Displayed as: col proportions (percent)
#>
#>                female: Male female: Female
#> senior: FALSE         78.51           70.6
#> senior: TRUE          21.49           29.4
```

`scf_ratio()` estimates the ratio of two totals, which is how the Fed computes shares and ratios such as wealth to income. `scf_corr()` gives the weighted correlation of two variables.

```
scf_ratio(scf2022, ~networth, ~income, by = ~senior)
#> SCF Ratio Estimate: networth / income
#>
#>  group numerator denominator  estimate        se       min       max
#>  FALSE  networth      income  5.272581 0.5700223  5.117185  5.449823
#>   TRUE  networth      income 18.974150 6.2802824 15.868752 22.077372
scf_corr(scf2022, ~log_income, ~log_networth)
#> SCF Correlation Estimate
#> Variables: log_income and log_networth
#>
#>  correlation         se        t       df      p.value      min       max
#>    0.5021715 0.05640718 8.902616 707.0645 4.547223e-18 0.487138 0.5227601
```

2.4 4. Hypothesis Tests

`scf_ttest()` and `scf_prop_test()` pool results across implicates and use Rubin's degrees of freedom.

```
scf_ttest(scf2022, ~networth, mu = 250000)
#> SCF One-sample t-test
#> Alternative hypothesis: mean is not equal to 250,000
#>
#> Estimate: 1000482.40
#> Standard Error: 177855.80
#> t = 4.22, df = 1337.5, p < 1e-04 ***
#> CI (95%): [651575.70, 1349389.09]
scf_ttest(scf2022, ~networth, group = ~senior)
#> SCF Two-sample t-test
#> Alternative hypothesis: difference in means is not equal to 0
#>
#> Group means:
#>  group      mean
#>  FALSE  809777.2
#>   TRUE 1595858.8
#>
#> Estimate: -786081.60
#> Standard Error: 638289.89
#> t = -1.23, df = 285.3, p = 0.2191
#> CI (95%): [-2042435.93, 470272.74]
scf_prop_test(scf2022, ~senior, p = 0.25)
#>
#> One-sample proportion test
#> Null hypothesis: proportion = 0.25
#> Alternative hypothesis: two.sided
#> Confidence level: 95%
#>
#>  estimate std.error t.value       df p.value conf.low conf.high stars
#>    0.2423    0.0311 -0.2476 187808.7  0.8044   0.1813    0.3033
scf_prop_test(scf2022, ~rich, ~female)
#>
#> Two-sample proportion test
#> Null hypothesis: proportion difference = 0
#> Alternative hypothesis: two.sided
#> Confidence level: 95%
#>
#>  estimate std.error t.value      df   p.value conf.low conf.high stars
#>    0.1626     0.042  3.8694 23.4735 0.0007555   0.0758    0.2494   ***
#>
#> Estimated group proportions:
#>   group proportion
#>    Male     0.1842
#>  Female     0.0216
```

2.5 5. Regression Modeling

`scf_ols()`, `scf_glm()`, and `scf_logit()` fit replicate-weighted models to each implicate and pool coefficients via Rubin's Rules.

```
scf_ols(scf2022, log_networth ~ age + log_income)
#> OLS Regression Results (Multiply-Imputed SCF)
#> ------------------------------------------------
#>         term  estimate std.error t.value   p.value stars
#>  (Intercept) -16.34472   2.71579  -6.018 3.043e-09   ***
#>          age   0.08801   0.01307   6.736 2.165e-11   ***
#>   log_income   2.03060   0.22398   9.066 5.666e-18   ***
#>
#> Model Fit Statistics:
#>   Mean R-squared: 0.4035 (SD: 0.011)
#>   Mean AIC:       1022.948 (SD: 5.0322)
#>
#> Note: Implicate-level model objects are stored in `object$imps`
#>       Use `summary(object$imps[[1]])` to inspect them.
scf_logit(scf2022, rich ~ age + log_income, odds = TRUE)
#> Logistic Regression Results (Multiply-Imputed SCF)
#> ------------------------------------------------
#>         term estimate std.error t.value   p.value stars
#>  (Intercept)   0.0000    0.0000 -4.2626 5.032e-05   ***
#>          age   1.0964    0.0399  2.5263   0.01415     *
#>   log_income  14.1789    8.9108  4.2195 6.008e-05   ***
#>
#> Model Fit Diagnostics:
#>   Pseudo R-squared:  0.5069
#>   Mean AIC:          80.6407
#>
#> Notes:
#>  - Estimates are reported on the Odds Ratio scale.
#>  - Implicate-level models are stored in `object$imps`
scf_glm(scf2022, own ~ age, family = binomial())
#> Generalized Linear Model (Multiply-Imputed SCF)
#> ------------------------------------------------
#>         term estimate std.error t.value p.value stars
#>  (Intercept)   1.4508    0.6709  2.1625 0.03058     *
#>          age   0.0070    0.0127  0.5495 0.58265
#>
#> Model Fit Diagnostics:
#>   Pseudo R-squared: 0.002 (SD: 0.000)
#>   Mean AIC:         165 (SD: 0)
#>
#> Note: Coefficients pooled with Rubin's rules; fit statistics averaged
across implicates.
#>       Inspect individual models via `object$models[[i]]`.
```

`scf_regtable()` formats results from one or more models for publication:

```
m1 <- scf_ols(scf2022, log_networth ~ age)
m2 <- scf_ols(scf2022, log_networth ~ age + log_income)
scf_regtable(m1, m2, model.names = c("Model 1", "Model 2"), digits = 3)
#> Term                  Model 1             Model 2
#> (Intercept)  6.551*** (0.918)  -16.345*** (2.716)
#> age          0.082*** (0.015)    0.088*** (0.013)
#> log_income                --    2.031*** (0.224)
#> ------------------------------------------------
#> N                        200                 200
#> R2                     0.133               0.404
#> AIC                     1096                1023
```

2.6 6. Quantile Regression

OLS models the conditional mean, which is sensitive to outliers and skew. Wealth and income distributions in the SCF are highly right-skewed: a small number of high-wealth households can dominate mean estimates. Quantile regression estimates the conditional quantile of the outcome at a user-specified probability `tau`. Estimating several quantiles shows how a predictor's effect differs across the distribution.

`scf_quantreg()` fits `quantreg::rq()` with SCF final sampling weights to each implicate, then pools coefficients and variance-covariance matrices across implicates via `scf_MIcombine()`.

```
# Median regression
m_med <- scf_quantreg(scf2022, log_networth ~ age + senior, tau = 0.50)
print(m_med)
#> Quantile Regression Results (tau = 0.50, Multiply-Imputed SCF)
#> ---------------------------------------------------------------------
#>         term estimate std.error t.value   p.value stars
#>  (Intercept)   8.2337    0.8347  9.8646 < 2.2e-16   ***
#>          age   0.0772    0.0168  4.6051 4.161e-06   ***
#>   seniorTRUE  -1.6873    0.5053 -3.3391 0.0008635   ***
#>
#> SE method: replicate | Implicates pooled: 5
#> R1(tau):   0.0587    R1 adj:  0.0444
#> Implicate-level rq objects stored in `object$models`.
```

```
# 75th percentile, with fast standard errors (see below)
m_75 <- scf_quantreg(scf2022, log_networth ~ age + senior, tau = 0.75,
                     se = "nid")
summary(m_75)
#> SCF Quantile Regression Summary (tau = 0.75)
#> ---------------------------------------------------------------------
#> Pooled Coefficient Estimates:
#>         term estimate std.error t.value   p.value stars
#>  (Intercept)   9.4118    0.9856  9.5494 < 2.2e-16   ***
#>          age   0.0761    0.0185  4.1243 3.856e-05   ***
#>   seniorTRUE  -1.7278    0.6220 -2.7776  0.005484    **
#>
#> Quantile:        0.75
#> SE method:       nid
#> Implicates used: 5
#>
#> Goodness of Fit (Koenker-Machado, 1999):
#>   Rho (full model):  163.7557
#>   Rho (null model):  173.5444
#>   R1(tau):           0.0564
#>   R1(tau) adjusted:  0.0420
#>   Mean N (implicates): 200
#>   Note: R1 is a local fit measure at tau; it is not a global
#>   summary of fit across the conditional distribution.
#>   R1 adjusted uses a df-penalty not derived from asymptotic
#>   theory; interpret it descriptively.
#>
#> Call:
#> scf_quantreg(object = scf2022, formula = log_networth ~ age +
#>     senior, tau = 0.75, se = "nid")
```

The `se` argument controls within-implicate variance estimation. All methods feed a covariance matrix into `scf_MIcombine()`, so between-implicate (imputation) variance is always incorporated.

- `"replicate"` (default): re-fits the model with each of the SCF's 999 replicate weights in each implicate, via `survey::withReplicates()`. The variance is the spread of the replicate coefficients around their mean, as the Fed computes it. This is the only method that uses the SCF's sample design. It refits the model about 5,000 times, so it can take several minutes on the full data.
- `"nid"`: non-iid sandwich estimator. Allows the conditional sparsity (the density of the error distribution at the quantile) to vary across observations.
- `"iid"`: the Koenker-Bassett (1978) covariance formula, which assumes the error distribution is the same for all observations. Fastest option.

- `"ker"`: kernel density estimate of the conditional sparsity.
- `"boot"`: pairs bootstrap.

The last four come from `quantreg::summary.rq()` and treat the weighted sample as a simple random sample, so they ignore the SCF's design. They are useful for quick exploration; report results with `"replicate"`.

```
# Fast standard errors while exploring
m_quick <- scf_quantreg(scf2022, log_networth ~ age + senior,
                        tau = 0.50, se = "nid")
```

A note on design: the `survey` package provides `svyquantile()` for estimating marginal quantiles of a single variable, but it has no function for regression quantiles (conditional quantiles given covariates). `scf_quantreg()` therefore uses `quantreg::rq()` for point estimation, with `survey::withReplicates()` providing the design-based variance wrapper for the `"replicate"` path.

Koenker and Bassett (1978) established the asymptotic theory for the unweighted, i.i.d. case. Applying `rq()` with survey sampling weights extends the estimator to probability-weighted samples, which is standard empirical practice; the `"iid"` and `"nid"` standard error formulas extend correspondingly to the weighted design matrix `X'WX`. The `"replicate"` option sidesteps this by using the SCF's own replication scheme to estimate variance directly, without relying on distributional assumptions about the errors.

Results are compatible with `scf_regtable()`:

```
scf_regtable(m_med, m_75,
             model.names = c("Median", "75th Pct"),
             digits = 3)
#> Term                    Median          75th Pct
#> (Intercept)   8.234*** (0.835)  9.412*** (0.986)
#> age           0.077*** (0.017)  0.076*** (0.018)
#> seniorTRUE   -1.687*** (0.505)  -1.728** (0.622)
#> ------------------------------------------------
#> N                         200               200
#> Tau                      0.50              0.75
#> R1                      0.059             0.056
#> R1(adj)                 0.044             0.042
```

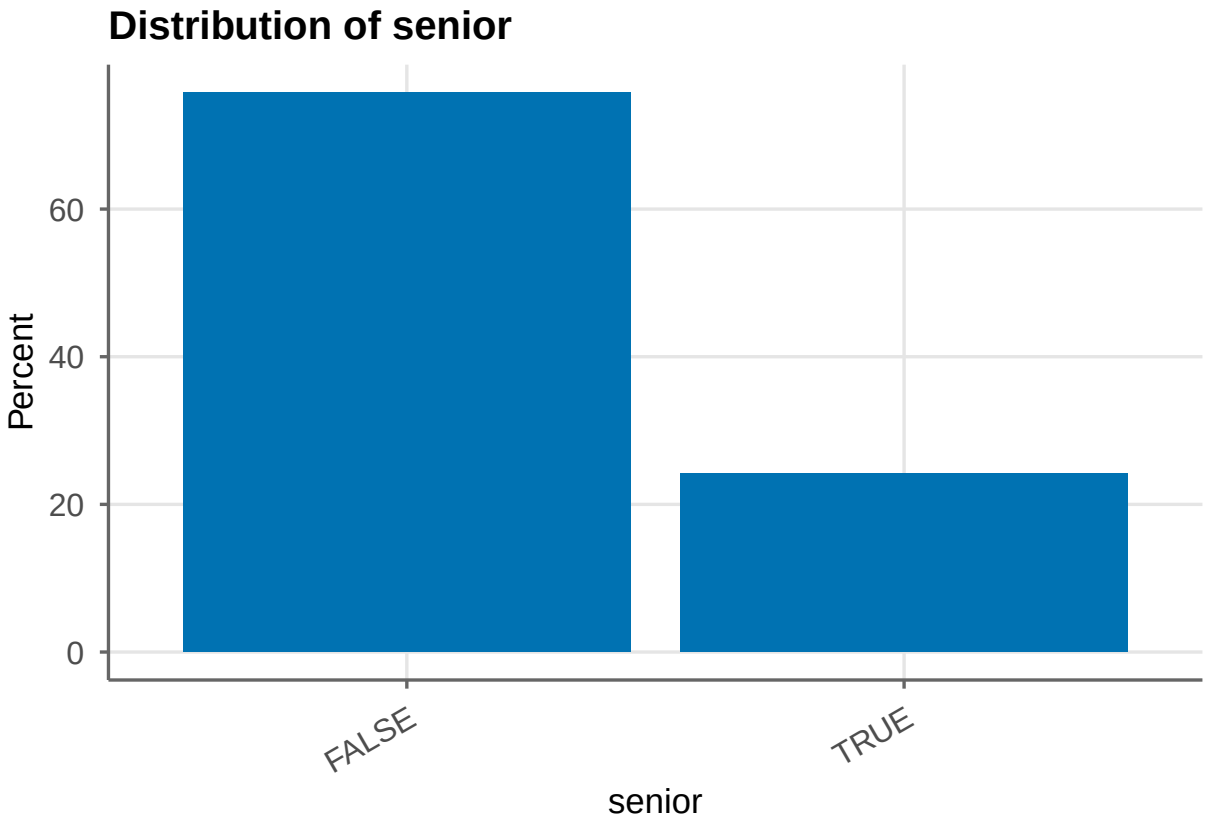
To study how associations vary across the outcome distribution, estimate the same model at multiple quantiles and compare:

```
taus <- c(0.25, 0.50, 0.75, 0.90)
models <- lapply(taus, function(t) {
  scf_quantreg(scf2022, log_networth ~ age + senior, tau = t)
})
names(models) <- paste0("tau=", taus)
do.call(scf_regtable, c(models, list(digits = 3)))
```

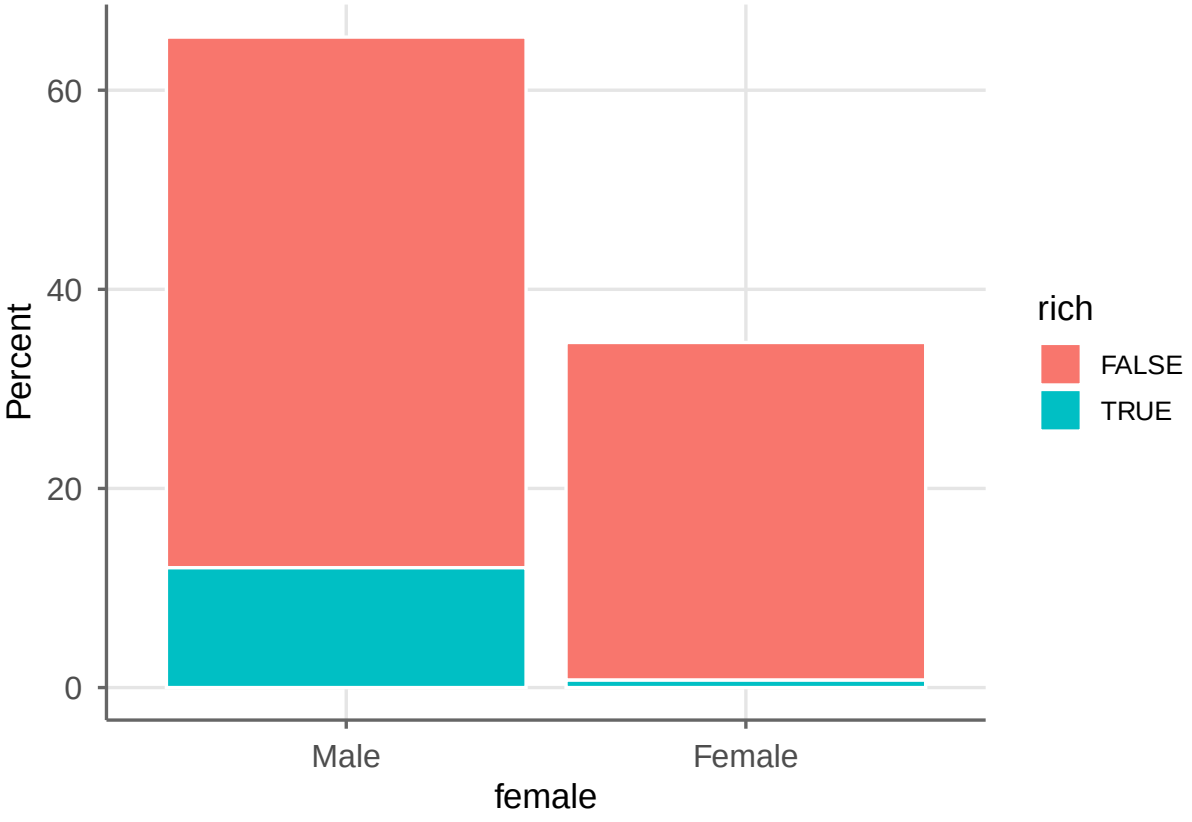
2.7 7. Visualization

`scf` plotting functions account for survey weights and multiply-imputed data.

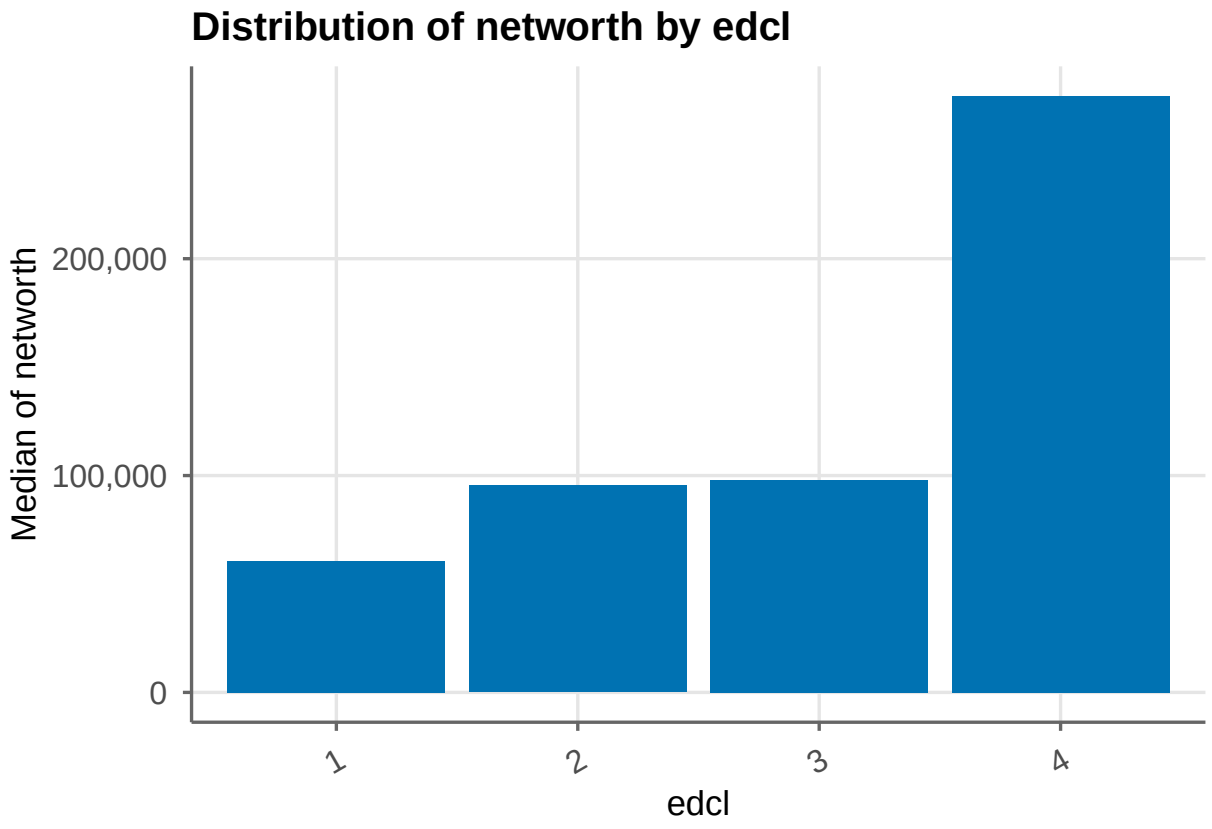
```
scf_plot_dbar(scf2022, ~senior)
```



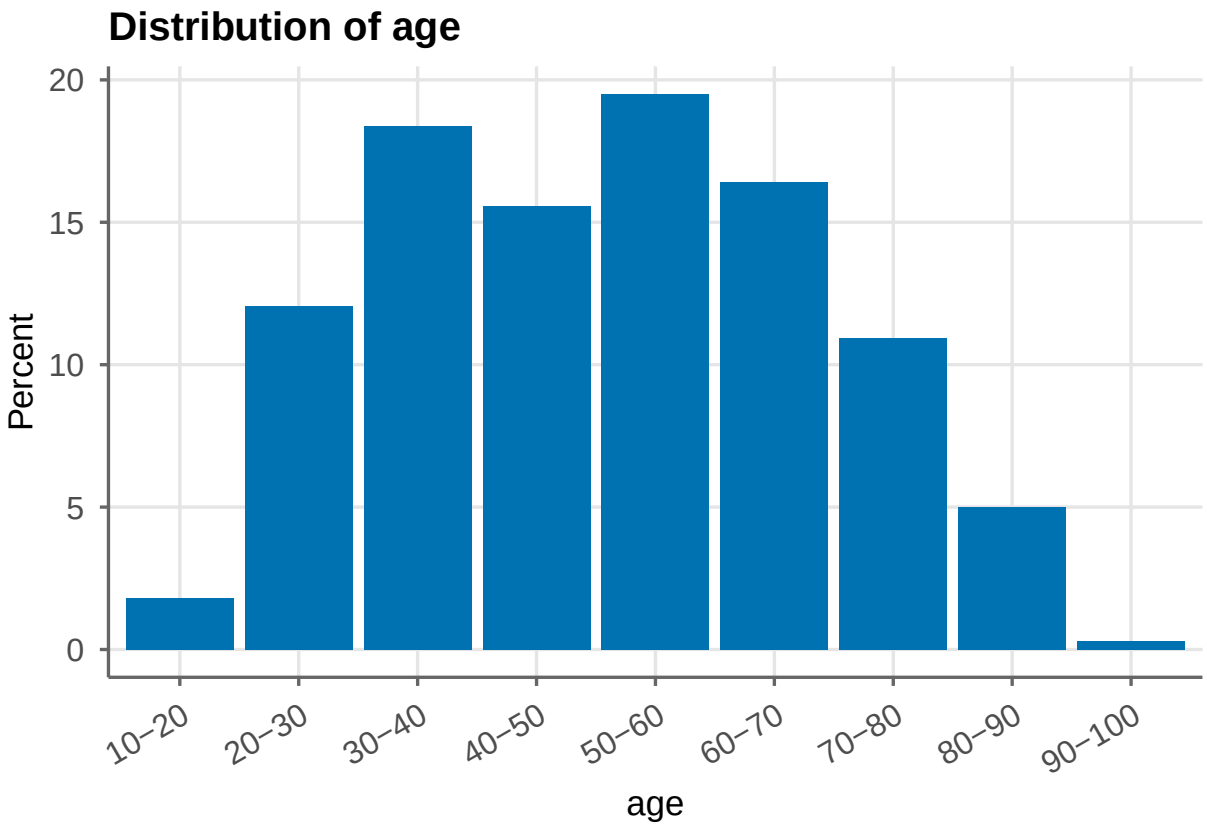

```
scf_plot_bbar(scf2022, ~female, ~rich, scale = "percent")
```



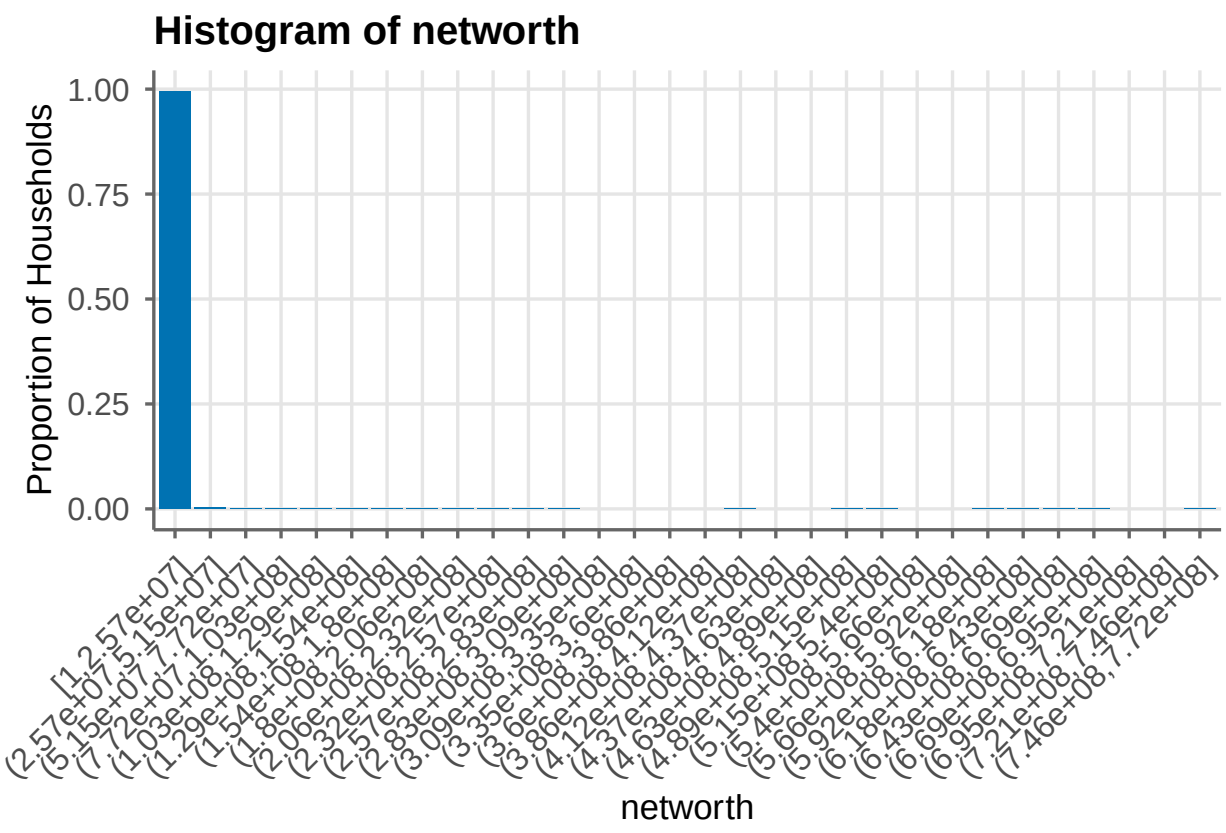

```
scf_plot_cbar(scf2022, ~networth, ~edcl, stat = "median")
```



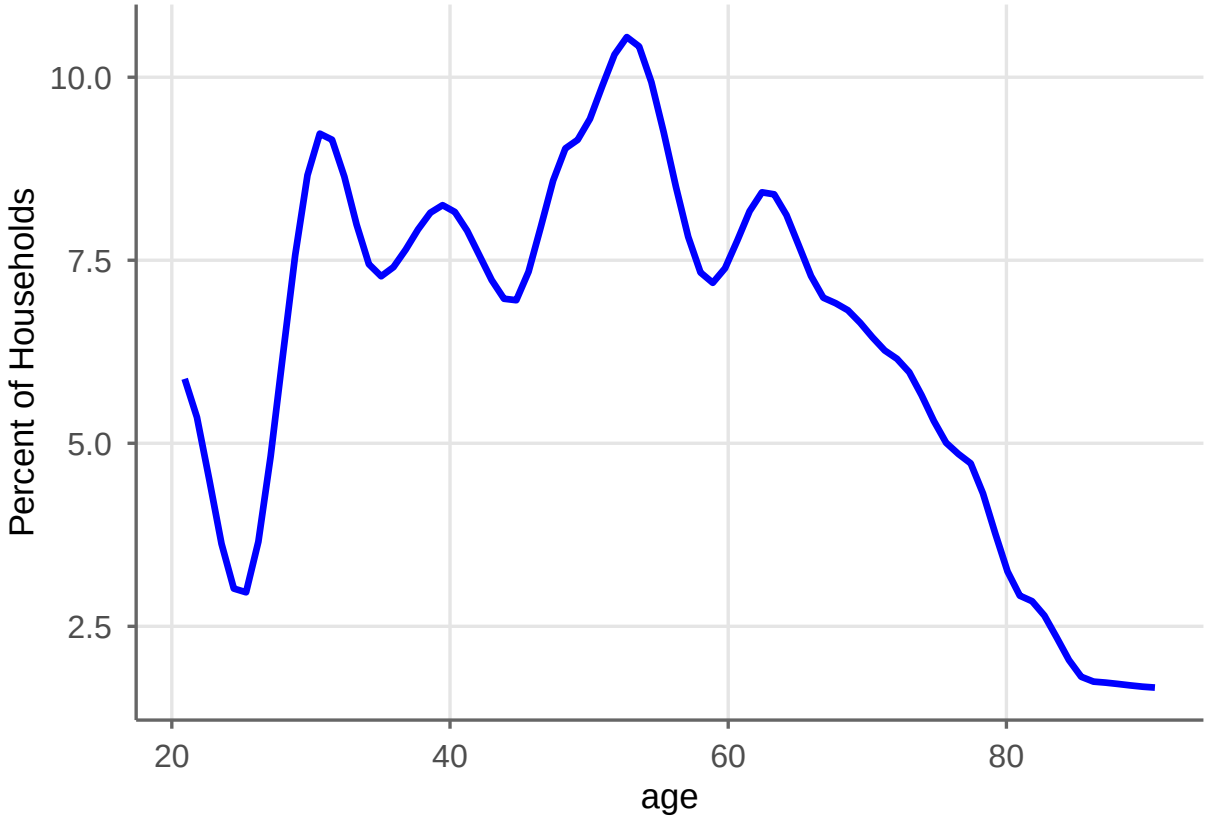

```
scf_plot_dist(scf2022, ~age, bins = 10)
```



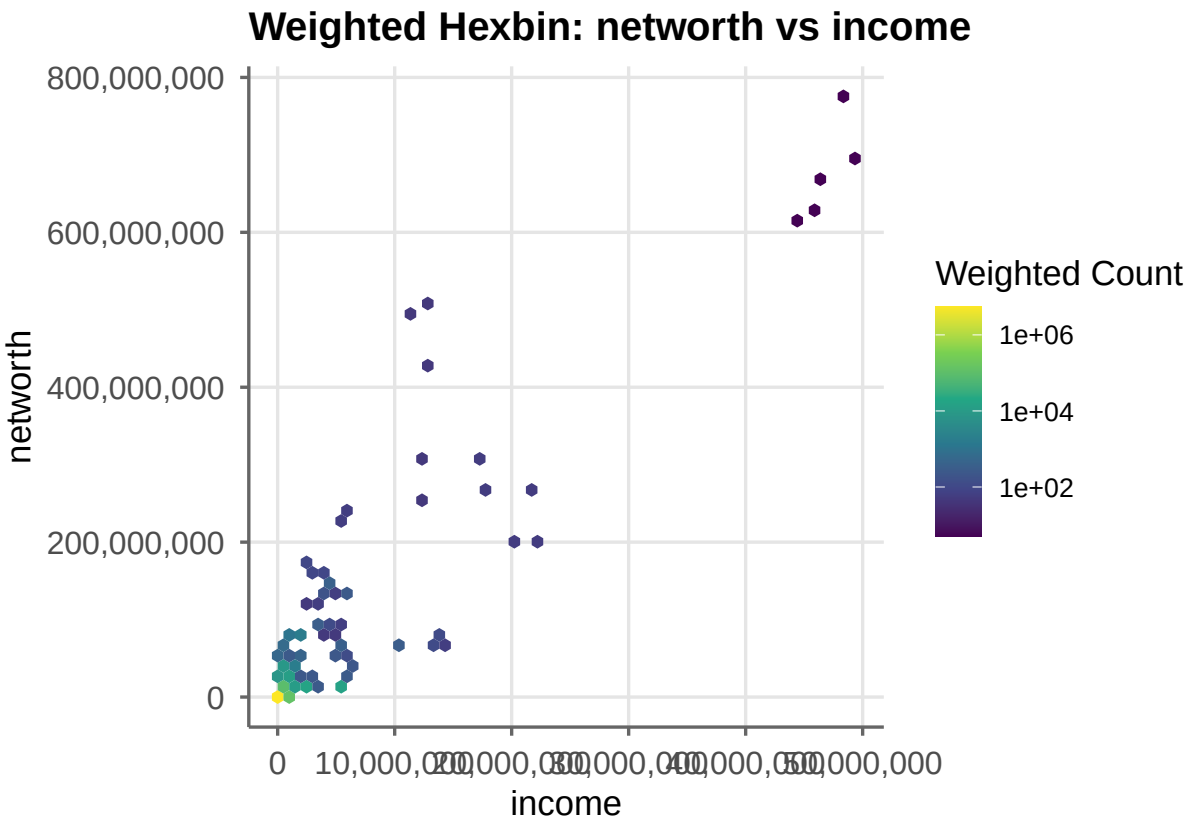

```
scf_plot_hist(scf2022, ~networth)
```




```
scf_plot_smooth(scf2022, ~age)
```




```
scf_plot_hex(scf2022, ~income, ~networth)
```



2.8 8. Inspecting Implicates

Use `scf_implicates()` to retrieve implicate-level estimates from any `scf_*` result for sensitivity analysis or custom pooling.

```
freq_table <- scf_freq(scf2022, ~rich)
scf_implicates(freq_table, long = TRUE)
#>            implicate group category       est          var  estimate
se
#> richFALSE          1   All    FALSE 0.8730810 0.0004431619 0.8730810
0.02105141
#> richTRUE           1   All     TRUE 0.1269190 0.0004431619 0.1269190
0.02105141
#> richFALSE1         2   All    FALSE 0.8531922 0.0004587259 0.8531922
0.02141789
#> richTRUE1          2   All     TRUE 0.1468078 0.0004587259 0.1468078
0.02141789
#> richFALSE2         3   All    FALSE 0.8725839 0.0004235772 0.8725839
0.02058099
#> richTRUE2          3   All     TRUE 0.1274161 0.0004235772 0.1274161
0.02058099
#> richFALSE3         4   All    FALSE 0.8794327 0.0003726167 0.8794327
0.01930328
#> richTRUE3          4   All     TRUE 0.1205673 0.0003726167 0.1205673
0.01930328
#> richFALSE4         5   All    FALSE 0.8827906 0.0004026946 0.8827906
0.02006725
#> richTRUE4          5   All     TRUE 0.1172094 0.0004026946 0.1172094
0.02006725
#>                 lower     upper         cv
#> richFALSE  0.83182025 0.9143418 0.02411164
#> richTRUE   0.08565822 0.1681798 0.16586495
#> richFALSE1 0.81121315 0.8951713 0.02510324
#> richTRUE1  0.10482873 0.1887868 0.14589068
#> richFALSE2 0.83224516 0.9129226 0.02358626
#> richTRUE2  0.08707735 0.1677548 0.16152583
#> richFALSE3 0.84159826 0.9172671 0.02194970
#> richTRUE3  0.08273288 0.1584017 0.16010378
#> richFALSE4 0.84345883 0.9221225 0.02273161
#> richTRUE4  0.07787754 0.1565412 0.17120863
```

Implicate-level regression model objects are stored directly in the result:

```
m <- scf_ols(scf2022, log_networth ~ age)
summary(m$imps[[1]])   # first implicate svyglm object
```

The usual model methods work on the pooled result: `coef()`, `SE()`, `vcov()`, `confint()`, `AIC()`, `predict()`, and `residuals()`. `SE()` also returns the standard errors of descriptive results.

```
SE(m2)
#> (Intercept)         age  log_income
#>  2.71579046  0.01306581  0.22397807
confint(m2)
#>                    2.5 %     97.5 %
#> (Intercept) -21.67817388 -11.011265
#> age           0.06238746   0.113638
#> log_income    1.59027041   2.470924
SE(scf_mean(scf2022, ~networth, by = ~senior))
#>    FALSE     TRUE
#> 123986.1 619767.0
```

3 Learn More

- Cohen JN. *Analyzing the Survey of Consumer Finances with scf: Methodology, User Guide, and Package Validation*. Working paper, CUNY Queens College; 2026. https://academicworks.cuny.edu/qc_pubs/695/
- Federal Reserve Board. *Survey of Consumer Finances*. https://www.federalreserve.gov/econres/aboutscf.htm
- Koenker R, Bassett G. Regression quantiles. *Econometrica*. 1978;46(1):33-50.
- Rubin DB. *Multiple Imputation for Nonresponse in Surveys*. Wiley; 1987.